HTTP Protocol — Lecture Notes

1. Core Properties of HTTP

HTTP is the primary protocol for client-server communication on the web. Two foundational ideas govern it:

Statelessness

- The server retains **no memory** of past requests. Each request is self-contained.
- Every request must carry all necessary context: auth tokens, session info, headers, etc.
- **Benefits:** Simplicity (no session storage on server), Scalability (any server can handle any request), Fault tolerance (server crash doesn't corrupt client state).
- **Workaround:** Developers implement state via cookies, sessions, or tokens when continuity is needed (e.g., login sessions).

Client-Server Model

- **Client** (browser/app) always initiates; sends request with URL, headers, body.
- **Server** hosts resources (APIs, web pages, files), processes requests, sends responses.
- HTTP communication is **always client-initiated**.

> `HTTP ≈ HTTPS` for conceptual purposes — HTTPS adds TLS encryption on top but the underlying principles are identical.

2. HTTP over TCP

HTTP uses **TCP** as the transport layer (OSI Layer 4), leveraging its reliability guarantees. HTTP itself only requires a reliable transport — TCP fits because it's connection-oriented and doesn't silently drop messages.

```
OSI Model — 7 Layers:
Layer 7 — Application   (HTTP, DNS, FTP)      ← Backend engineers work
here
Layer 6 — Presentation  (Encryption, TLS)
Layer 5 — Session       (Session management)
Layer 4 — Transport     (TCP, UDP)             ← HTTP uses TCP
Layer 3 — Network       (IP, routing)
```

```
Layer 2 – Data Link     (MAC, Ethernet frames)
Layer 1 – Physical      (Cables, signals)
```

Data flows **down** all 7 layers on the sender side, travels over the physical link, and flows **back up** all 7 layers on the receiver side.

---

3. Evolution of HTTP

Version	Key Feature
HTTP/1.0	New TCP connection per request — inefficient
HTTP/1.1	**Persistent connections** (connection reuse), chunked transfer, better caching
HTTP/2.0	**Multiplexing** (multiple req/res over one connection), binary framing, HPACK header compression, server push
HTTP/3.0	Built on **QUIC** (UDP-based), faster handshake, lower latency, better packet loss handling, no head-of-line blocking

---

4. HTTP Message Structure

Request Message

```
PUT /api/users/12345 HTTP/1.1
Host: example.com
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) ...
Content-Type: application/json
Content-Length: 123
Authorization: Bearer eyJhbGciOi...
Accept: application/json
Accept-Encoding: gzip, deflate, br
Connection: keep-alive
Referer: https://example.com/dashboard
Cookie: sessionId=abc123xyz456; lang=en-US
                              ← blank line separates headers from body
{
  "firstName": "John",
  "lastName": "Doe",
  "email": "john.doe@example.com",
  "age": 30
}
```

Response Message

```
HTTP/1.1 200 OK
Date: Fri, 20 Sep 2024 12:00:00 GMT
Content-Type: application/json
Content-Length: 85
Server: Apache/2.4.41 (Ubuntu)
Cache-Control: no-store
X-Request-ID: abcdef123456
Strict-Transport-Security: max-age=31536000; includeSubDomains; preload
Set-Cookie: sessionId=abc123xyz456; Path=/; Secure; HttpOnly
Vary: Accept-Encoding
Connection: keep-alive
                              ← blank line
{
  "message": "User updated successfully",
  "userId": 12345,
  "status": "success"
}
```

> Notable headers from the example: `X-Request-ID` is a custom header for tracing/debugging individual requests across logs. `Vary` tells caches which request headers affect the response.

---

5. HTTP Headers

Headers are **key-value metadata** attached to requests/responses — analogous to the label on a parcel (visible externally, no need to open the package).

Categories

Request Headers — Client → Server context

- `User-Agent`: Identifies client type (browser, Postman, mobile app)
- `Authorization`: Credentials (e.g., `Bearer <JWT>`)
- `Accept`: Desired response format (`application/json`, `text/html`)
- `Cookie`: Sends stored cookies to the server (session IDs, preferences)
- `Referer`: URL of the page that initiated the request (useful for analytics/security)

General Headers — Present in both

- `Date`, `Cache-Control` (`no-cache`, `max-age`), `Connection` (`keep-alive`, `close`)

Representation Headers — Describe the body

- `Content-Type`: Media type of body (`application/json`, `text/html`)
- `Content-Length`: Size in bytes

- `Content-Encoding`: Compression (`gzip`, `deflate`)
- `ETag`: Hash/identifier for caching

Security Headers

- `Strict-Transport-Security (HSTS)`: Force HTTPS, prevent protocol downgrade
- `Content-Security-Policy (CSP)`: Restrict JS/CSS/image sources → prevents XSS
- `X-Frame-Options`: Prevent iframe embedding → prevents clickjacking
- `X-Content-Type-Options`: Prevent MIME-type sniffing
- `Set-Cookie` with `HttpOnly` / `Secure` flags: Protect cookies

Key Properties

- **Extensibility:** Headers can be added/customized without changing the protocol.
- **Remote control:** Headers let clients instruct server behavior (format, caching, auth).

---

6. HTTP Methods

Method	Intent	Has Body	Notes
`GET`	Fetch resource	No	Must not modify server state
`POST`	Create resource	Yes	Non-idempotent
`PATCH`	Partial update	Yes	Selective replacement; preferred over PUT
`PUT`	Full replacement	Yes	Completely replaces the resource
`DELETE`	Delete resource	No	—
`OPTIONS`	Query server capabilities	No	Used in CORS preflight

Idempotency

An HTTP method is **idempotent** if calling it N times produces the same result as calling it once.

- **Idempotent:** `GET`, `PUT`, `DELETE`
- **Non-idempotent:** `POST` (each call may create a new resource)

---

7. CORS and the OPTIONS Method

Same-Origin Policy: Browsers restrict web pages from making requests to a different domain/port than the one serving the page.

CORS (Cross-Origin Resource Sharing): A server-side mechanism to selectively allow cross-origin requests.

Simple Request Flow

Applies when: method is `GET` / `POST` / `HEAD` + no non-simple headers + simple `Content-Type`.

```
Client (example.com) ——— GET + Origin: example.com ———► Server
(api.example.com)
                     ◄——— 200 + Access-Control-Allow-Origin: example.com
————
Browser checks header → allows or blocks response
```

If `Access-Control-Allow-Origin` is absent → **CORS error**, response blocked.

Preflight Request Flow

Triggered when **any one** of:

1. Method is `PUT`, `DELETE`, etc. (not GET/POST/HEAD)
2. Request includes non-simple headers (e.g., `Authorization`, custom headers)
3. `Content-Type` is `application/json` (most API calls qualify)

Flow:

```
1. Browser sends OPTIONS request (preflight):
   OPTIONS /resource HTTP/1.1
   Origin: example.com
   Access-Control-Request-Method: PUT
   Access-Control-Request-Headers: Authorization, Content-Type

2. Server responds (204 No Content):
   Access-Control-Allow-Origin: example.com
   Access-Control-Allow-Methods: GET, POST, PUT, DELETE
   Access-Control-Allow-Headers: Authorization, Content-Type
   Access-Control-Max-Age: 86400   ← cache preflight for 24h

3. Browser sends actual request only if preflight succeeded.
```

`Access-Control-Max-Age` avoids re-sending preflight for every request — saves bandwidth.

---

8. HTTP Status Codes

1xx — Informational

- `100 Continue`: Server received headers, client can send body (used in large uploads)
- `101 Switching Protocols`: Upgrade to WebSocket

2xx — Success

Code	Meaning	When to use
`200 OK`	Request succeeded	GET responses, general success
`201 Created`	Resource created	POST/PUT that creates a new entity
`204 No Content`	Success, no body	DELETE, preflight OPTIONS response

3xx — Redirection

Code	Meaning	When to use
`301 Moved Permanently`	Resource moved; update URL	Route migrations (backwards compatibility)
`302 Found`	Temporary redirect	A/B tests, campaigns
`304 Not Modified`	Use cached version	Cache validation with ETag/Last-Modified

4xx — Client Errors

Code	Meaning	When to use
`400 Bad Request`	Invalid input/format	Wrong data type, malformed body
`401 Unauthorized`	Not authenticated	Missing/expired JWT or token
`403 Forbidden`	Authenticated but lacks permission	User A trying to delete User B's resource
`404 Not Found`	Resource doesn't exist	Invalid URL or deleted resource
`405 Method Not Allowed`	Wrong HTTP method for route	PATCH on a GET-only route
`409 Conflict`	State conflict	Duplicate resource (e.g., folder name taken)
`429 Too Many Requests`	Rate limit exceeded	After N requests in time window

5xx — Server Errors

Code	Meaning	When to use
`500 Internal Server Error`	Unhandled exception	Unexpected crash
`501 Not Implemented`	Feature not yet available	Future endpoint placeholder
`502 Bad Gateway`	Upstream returned invalid response	Nginx/proxy upstream failure
`503 Service Unavailable`	Server down/overloaded	Maintenance, traffic spike
`504 Gateway Timeout`	Upstream didn't respond in time	Nginx timeout from origin server

---

9. HTTP Caching

Reduces redundant data transfer by allowing clients to reuse cached responses.

Key Headers

Header	Direction	Purpose
`Cache-Control: max-age=N`	Response	Cache is valid for N seconds
`ETag: "hash"`	Response	Unique identifier (hash) of the resource
`Last-Modified: <date>`	Response	Timestamp of last modification
`If-None-Match: "hash"`	Request	Send new data only if ETag differs
`If-Modified-Since: <date>`	Request	Send new data only if modified after date

Flow

```
1st Request:  GET /resource
Response:     200 OK + body + ETag: "abc" + Cache-Control: max-age=10

2nd Request (within TTL):
              GET /resource
              If-None-Match: "abc"
              If-Modified-Since: <timestamp>
Response:     304 Not Modified (no body, use cache)

After update:
```

```
                GET /resource + If-None-Match: "abc"
Response:       200 OK + new body + ETag: "xyz"
```

> **Note:** Modern apps often prefer client-side cache libraries (e.g., React Query) over manual HTTP cache headers — more control, less risk of stale data from missed ETag updates.

---

10. Content Negotiation

Mechanism for client and server to agree on the format of exchanged data.

Header	Purpose	Example
`Accept`	Desired media type	`application/json`, `text/xml`
`Accept-Language`	Preferred language	`en`, `es`
`Accept-Encoding`	Supported compression	`gzip`, `deflate`, `br`
`Content-Type`	Actual body format	`application/json`
`Content-Encoding`	Applied compression	`gzip`

The server reads the `Accept-*` headers and responds in the best matching format.

HTTP Compression

Compression drastically reduces payload size:

- Example from lecture: 11,000-entry file → 3.8 MB (gzip) vs 26 MB (uncompressed)
- Flow: Client sends `Accept-Encoding: gzip` → Server compresses + responds with `Content-Encoding: gzip` → Browser decompresses transparently.
- **Tradeoff:** Gzip adds a small CPU overhead on both ends (compress on server, decompress in browser), but the savings in network transfer time far outweigh this cost — especially on slow/mobile connections.
- `Vary: Accept-Encoding` response header tells caches that the response may differ based on the client's encoding preference, preventing a compressed response from being served to a client that can't decompress it.

---

11. Persistent Connections & Keep-Alive

Version	Default Behavior
HTTP/1.0	New TCP connection per request (slow)
HTTP/1.1+	Persistent by default — connection reused across requests

- `Connection: keep-alive` header can explicitly request connection reuse, with optional `timeout` and `max` parameters.
- `Connection: close` forces connection teardown after response (HTTP/1.0 behavior).
- In practice, defaults work fine — no manual configuration needed.

---

12. Large Data Transfer

Multipart Requests (Client → Server)

Used for file uploads. Binary data is sent in **parts** separated by a boundary delimiter.

```
Content-Type: multipart/form-data; boundary=----FormBoundary7MA4YWxk
```

The `boundary` string acts as delimiter between parts in the request body.

Chunked Transfer / Server-Sent Events (Server → Client)

Used to stream large responses progressively.

Key headers:

```
Content-Type: text/event-stream
Connection: keep-alive
```

Client receives and appends chunks until the stream ends. Useful for large files, live data, LLM streaming, etc.

---

13. SSL / TLS / HTTPS — Quick Reference

Term	Description
SSL	Original encryption protocol — now deprecated (security flaws)
TLS	Modern replacement for SSL; current standard is TLS 1.3
HTTPS	HTTP + TLS encryption; protects data in transit from interception

TLS uses **certificates** to authenticate the server and establish an encrypted channel. Relevant at the network layer — as an application developer, enabling HTTPS (via a reverse proxy or cloud provider) is typically sufficient.

Quick Revision Checklist

- [ ] Statelessness: what it means, why it's beneficial, how state is faked (cookies/tokens)
- [ ] HTTP message anatomy: method, URL, version, headers, blank line, body
- [ ] Header categories: request, general, representation, security
- [ ] GET/POST/PATCH/PUT/DELETE semantics + idempotency
- [ ] CORS: same-origin policy → simple flow vs preflight flow → OPTIONS method
- [ ] Status codes: 2xx/3xx/4xx/5xx common ones and when to use each
- [ ] Caching: ETag + Cache-Control + 304 flow
- [ ] Content negotiation: Accept headers + compression benefit
- [ ] Persistent connections: HTTP/1.1 default, keep-alive header
- [ ] Large transfers: multipart (upload) vs chunked/SSE (download)
- [ ] HTTPS = HTTP + TLS encryption